Exploring Discontinuity Effects on Mechanical Design

By: Aaradhaya Sutrave

December 5, 2025

Table of Contents:

1. Literature Review **3**

1.1. Article 1: Stiffness degradation failure mechanisms and probabilistic fatigue life analysis of TA1 clinched joints 3

1.2. Article 2: Crystal-twinning inspired lattice metamaterial for high stiffness, strength, and toughness 5

2. Stepped Shaft Design Analysis **7**

2.1. Introduction 7

2.1.1. Problem Description 7

2.1.2. Design Requirements 7

2.1.3. Given Data and Assumptions 8

2.2. Analysis 9

2.2.1. Stiffness Analysis 9

2.2.2. Strength Analysis 11

2.2.3. Fatigue Analysis 12

2.2.4. FEA Analysis 14

2.2.5. Sensitivity 16

2.2.6. Design Tradeoff for 2nd Material 19

2.3. Conclusion 22

2.4. Design Justification 24

2.4.1. Proposed Changes 24

2.5. Design Standard 25

3. Failure Memo **26**

4. References **28**

5. Appendix **29**

Literature Review

Article 1: Stiffness degradation failure mechanisms and probabilistic fatigue life analysis of TA1 clinched joints

Authors: Yue Zhang, Xian Wang, Ruitao Peng, Changyou Xu, Tao Shi, and Kexi Chen

The fatigue behavior of TA1 titanium clinched joints is governed by a combination of geometric discontinuities, fretting wear, and evolving stiffness degradation. Under cyclic loading, the joints experience asymmetric deformation as the sheet assembly gradually folds into itself, resulting in uneven load transfer through the interlocking region. This deformation amplifies local contact stresses where the plates intersect, accelerating wear and promoting microcrack formation. Microscopic examinations reveal black wear and highly localized damage, indicating that slip and abrasion at the joint interface contribute significantly to fatigue initiation.

A major finding is the identification of three distinct stages of stiffness degradation. During the initial stabilization phase, stiffness decreases only little by little, reflecting elastic behavior and limited micro-damage at the contact surfaces. This transitions into a crack initiation phase marked by a sharper decline in stiffness, where localized plasticity and microcrack growth occur in the lower plate. The final stage is characterized by rapid stiffness loss as cracks propagate and combine, leading to catastrophic fracture. This clear correlation between stiffness evolution and crack progression highlights the value of stiffness monitoring as a diagnostic tool for fatigue assessment.

Load level plays a decisive role in determining fatigue life and damage progression. Higher load levels produced larger hysteresis loops and increased displacement with each cycle, indicating greater energy dissipation and accelerated wear. At lower loads, hysteresis curves remain narrow and stable for longer, reflecting a more balanced state where damage accumulation proceeds slowly. Nonetheless, across all load groups, failure consistently occurs through fracture of the lower plate, confirming it as the weakest region of the joint configuration.

The statistical evaluation of fatigue life using multiple probability models offers practical guidance for fatigue design with limited sample sizes. Analysis shows that extreme-value distributions, especially the Gumbel model, provide the most reliable representation of minimum-life behavior and best capture the scatter typical of fretting-dominated failures. This statistical outcome emphasizes that fatigue in clinched joints is controlled by local weakest-link mechanisms rather than uniform damage processes.

Overall, the study provides a detailed characterization of how mechanical degradation evolves in clinched joints and clarifies the mechanical pathways that lead to failure. The combined use of hysteresis analysis, stiffness tracking, fracture observations, and probabilistic modeling yields a comprehensive understanding of the fatigue mechanisms governing TA1

titanium sheet connections. The findings from this paper also guide how to evaluate stress concentrations and fatigue sensitivity in the step shaft. The article shows how stiffness degradation and crack initiation actually occur near geometric discontinuities.

<u>Article 2:</u> Crystal-twinning inspired lattice metamaterial for high stiffness, strength, and toughness

Authors: Kanghui Song, Dawei Li, Tingting Liu, Changdong Zhang, Yi Min Xie, and Wenhe Liao

The development of tetrahedral-based lattice structures integrated with topology optimization offers an effective strategy for creating lightweight components with tailored mechanical performance. The tetrahedral cell architecture provides structural stability, satisfies Maxwell's criterion for stretch dominated behavior, and allows for conformal meshing of complex shapes. These characteristics make it suitable for representing intermediate densities generated by optimization algorithms, addressing a common limitation in topology optimization where binary solid–void outputs often produce impractical slender regions.

A key contribution of the work lies in establishing a reliable stiffness–density relationship for these lattices and validating their near-isotropic behavior. By using pseudo-random tetrahedral meshing and maintaining an appropriate cell to structure size ratio, the lattice assemblies behave equivalently to homogenized isotropic materials for densities above approximately 0.2. Experimental compression tests on printed PLA lattice cubes confirm the accuracy of a third-order polynomial interpolation used to map density to stiffness in the optimization framework. This enables the lattices to be handled within SIMP without introducing anisotropic modeling errors.

The introduction of a porosity constraint creates a controlled way to adjust material distribution across the optimized structure. This constraint governs how much intermediate-density material is preserved, influencing both stiffness and failure behavior. Beam tests reveal that structures with intermediate porosity exhibit a favorable balance of mechanical attributes. These configurations avoid the brittle and stress concentration issues seen in fully solid or fully lattice beams and instead develop distributed damage zones with more stable failure progression. At the same time, they maintain stiffness levels comparable to those of solid–void optimized solutions.

The study also highlights manufacturing considerations that influence achievable relative densities and structural performance. Material extrusion printing imposes lower limits on lattice dimensions due to nozzle width and geometric accuracy. The chosen cell size and minimum printable truss thickness set a practical lower density bound of approximately 0.2. Despite these constraints, the printed beams closely match simulated predictions, validating both the mesh-generation strategy and the correspondence between the density field and the physical lattice geometry.

Failure mode observations provide further insight into how porosity affects structural integrity. Fully lattice structures tend to fail through tensile rupture at the lower surface, while

intermediate porosity samples exhibit mixed tensile–compressive damage that propagates in a more controlled manner. Structures with low porosity develop slender features susceptible to buckling. These varied responses demonstrate the ability to tune performance by selecting appropriate porosity ranges and highlight the importance of smooth density transitions.

Overall, the research establishes a coherent workflow for generating manufacturable, functionally graded lattice structures directly from topology-optimized density fields. By combining mechanical testing, numerical homogenization, and lattice generation techniques, the study demonstrates that intermediate-density lattices can provide improved stability and toughness while retaining high structural efficiency. The article's discussion of lattice induced stress redistribution informed how I considered alternate geometric modifications for improving fatigue performance. This also influenced my recommendation to increase the minimum diameter in Part 2.

Stepped Shaft Design Analysis

Introduction

A stepped shaft is subjected to combined bending and torsional loading while supporting a gear and transmitting mechanical power between two bearings. The shaft contains multiple diameter transitions along its 675 mm span, creating varying stiffness and strength characteristics that influence its performance. A 10 kN transverse load is applied at the gear location, generating bending stresses, while 25 kW of mechanical power is transmitted at 900 rpm, producing torsional shear.

The objective of this analysis is to evaluate the performance of the stepped shaft design using practices learned. The shaft will be analyzed for stiffness, static strength, and fatigue life to determine whether the baseline design meets all required performance criteria.

Design Requirements

The stepped shaft must satisfy the following requirements:

Stiffness Requirements:

- Maximum allowable slope at the gear locations:

$$\Theta_{gear} \leq 0.0005 \text{ rad}$$

- Maximum allowable slope at the bearing locations:

$$\Theta_{bearing} \leq 0.001 \text{ rad}$$

Static Strength Requirements:

- Minimum Factor of Safety against Yielding:

$$n_y \geq 1.5$$

Fatigue Requirements:

- Minimum Factor of Safety under fluctuating bending (Modified Goodman)

$$n_f \geq 2.0$$

These constraints ensure the shaft remains sufficiently stiff to maintain gear alignment, strong enough to avoid yielding under maximum loading, and durable enough to withstand long-term cyclic performance.

Given Data and Assumptions

Material Properties:
AISI 1020 Cold-Drawn Carbon Steel:

- Ultimate Tensile Strength: $S_u = 470\ MPa$
- Yield Strength: $S_y = 390\ MPa$
- Elastic Modulus: $E = 205\ GPa$

AISI 4140 Q&T Alloy Steel:

- Ultimate Tensile Strength: $S_u = 690\ MPa$
- Yield Strength: $S_y = 600\ MPa$
- Elastic Modulus: $E = 205\ GPa$

Both Materials have a density of $\rho = 7850\ kg/m^3$

Geometry:

Total Shaft Length: 375 mm

The shaft has nine stepped segments with varying diameters:

Segment	Length (mm)	Diameter (mm)
1	30	30
2	25	35
3	60	40
4	10	55
5	100	45
6	65	40
7	25	35
8	30	30
9	30	20

Loading Conditions:

- Transverse Load at Gear:

$$P = 10\ kN\ \ at\ \ x = 155mm$$

- Power Transmission:

$$P_{in} = 25\ kW\ and\ n = 900\ rpm$$

Boundary Conditions:

- The shaft is supported at two bearings (fixed)
- Deflection is zero at both supports
- Load is applied vertically downward
- Shaft is solid, axisymmetric and linearly elastic

Stiffness Analysis:

The stepped shaft analysis employs Macaulay's method to determine deflection and slope distributions using singularity functions. First, we need to find the reaction forces at both of the 30 mm diameter shafts.

$$R_B = \frac{P * a}{L} = \frac{10000 * 155}{375} = 4133.33\ N$$

$$R_A = P - R_B = 5866.67\ N$$

Using singularity functions, the bending moment distribution would look like:

$$M(x) = R_A\langle x\rangle^1 - P\langle x - 155\rangle^1$$

For each shaft segment, the moment of inertia is calculated by:

$$I = \frac{\pi d^4}{64}$$

Here is a table with the moment of inertia of each segment:

Segment	Moment of Inertia, I (mm^4)	Polar Moment of Inertia, J(mm^4)
1	39,760.8	79,521.6
2	73,669.6	147,339.2
3	125,663.7	251,327.4
4	449,596.9	899,193.8
5	201,964.6	403,929.2
6	125,663.7	251,327.4
7	73,669.6	147,339.2
8	39,760.8	79,521.6
9	7,854.0	15,708.0

Table 2: Moment of Inertia of each segment of the Stepped Shaft

The stepped nature make the singularity function for the M/I function formulated like this:

$$\frac{M}{I} = \frac{R_A}{I_1}\langle x\rangle^1 - \frac{P}{I_k}\langle x - 155\rangle^1 + \sum_{i=1}^{8}\left(\frac{1}{I_{i+1}} - \frac{1}{I_i}\right)[M(x_i)\langle x - x_i\rangle^0 + V(x_i)\langle x - x_i\rangle^1]$$

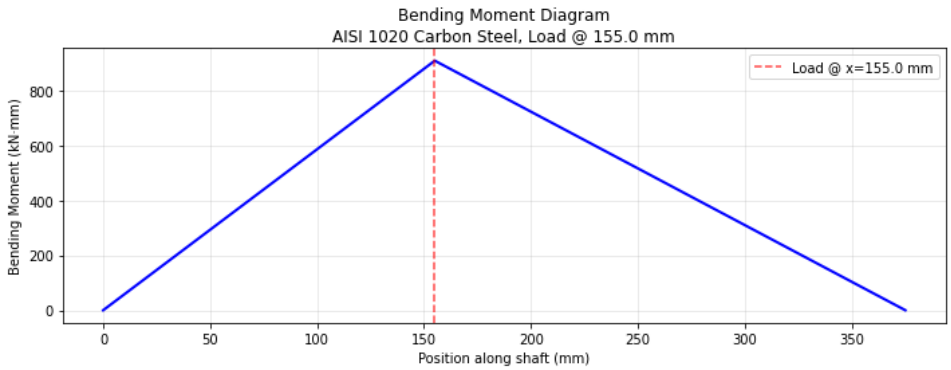


Figure 1: Bending Moment Diagram

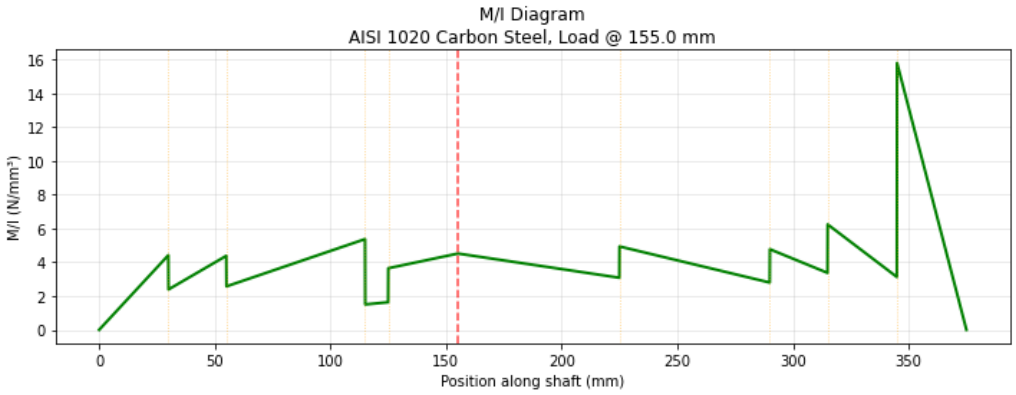


Figure 2: M/I Diagram for the 1st material

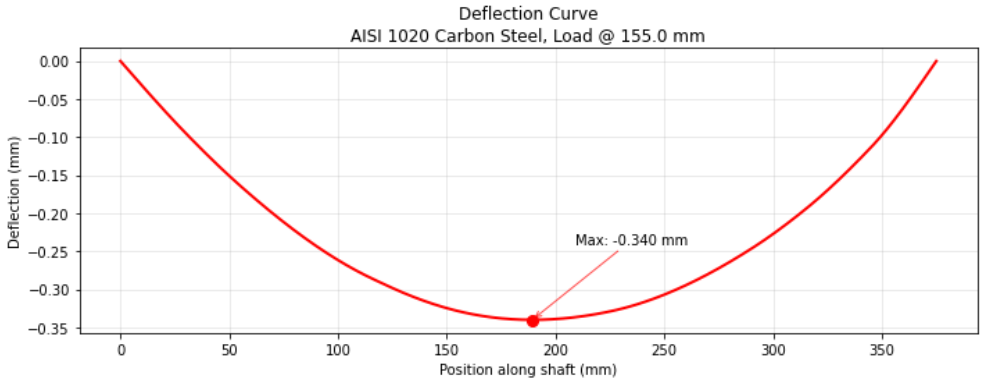


Figure 3: Displacement Curve for the 1st material

Where, I_k is the moment of inertia at the load segment (for example at segment 5, I=201964.6 mm^4), x_i are the step locations (30, 55,115…. 345 mm), and $M(x_i)$ and $V(x_i)$ are the bending moment and shear force at each step.

The singularity function is then integrated to find the slope of the deflection:

$$\theta(x) = \int \frac{M(x)}{EI(x)} dx + C_1$$

Then integrated a second time for deflection:

$$y(x) = \int \theta(x)\, dx \; + C_2$$

We then use Boundary conditions $y(0) = 0$ and $y(375) = 0$ to determine the integral constants C_1 *and* C_2. The integration is performed numerically to handles the piecewise nature of the M/I function.

To simplify the calculation, I used the code, provided in the appendix, to get an accurate solution. I used the assumptions that the 10 kN transverse load was 155 mm from the left of the step shaft,

there is a 265.28 N*m torque on the shaft and bearing supports at the 1st and 8th segments of the stepped shaft.

Critical Deflection and Slopes:

- Maximum Deflection: 0.340 mm downward at x=189.3 mm
- Slope at Left bearing: -0.003269 rad
- Slope at right bearing: 0.004164 rad
- Slope at gear location (x=175 mm, middle of key slot): -.000277 rad

Stiffness Factor of Safety Evaluation:

$$FoS_{bearing} = \frac{.001}{.004164} = .240$$

$$FoS_{gear} = \frac{.0005}{.000277} = 1.803$$

Conclusion: Both stiffness factors of safety are significantly less than 1.0, indicating that the shaft fails to meet the required stiffness constraints. The slopes at both bearings and the gear location exceed allowable limits by factors of 2.5-4.2.

Static Strength Analysis:

Stress Analysis Results:

- Critical Location: x=345 mm in the smallest diameter segment (d = 20.0 mm)
- Maximum bending stress: 276.3 MPa
- Max von Mises stress: 446.7 MPa
- Torsional Stress: 202.6 MPa

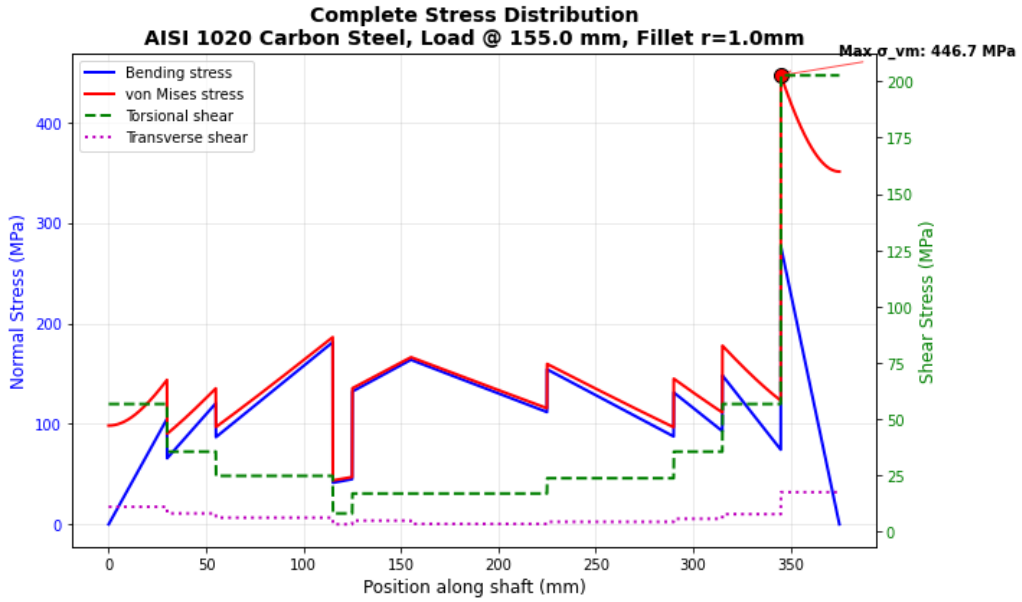


Figure 4: Stress distribution

Static Factor of Safety:

$$n_y = \frac{S_y}{\sigma_{vm}} = .87$$

Since the static factor of safety is under the requirement of $n_y \geq 1.5$, so the stepped shaft fails due to yielding. The critical stress occurs in the smallest diameter segment (20 mm), which experiences the highest combined stress despite not having the maximum bending moment, due to torsional loading.

Fatigue Analysis

Endurance Limit calculation:

$$S'_e = .5 * S_u = .5 * 470 = 235\ MPa$$

Marin Modifying Factors (for AISI 1020 Cold-drawn):

- Surface factor (k_a) = 0.9
- Size factor (k_b) = 0.85
- Load factor (k_c) = 1.0
- Temperature factor (k_d) = 1.0
- Reliability factor (k_e) = .868
- Miscellaneous (k_f) = 1.0

$$S_e = S'_e * k_a * k_b * k_c * k_d * k_e * k_f = 156.0\ MPa$$

Stress components at Critical Location (x = 345 mm):

- Alternating bending stress (fully reversed): $\sigma_a = 157.9\ MPa$
- Mean torsional stress (steady): $\tau_m = 168.9\ MPa$

Von Mises alternating and mean stresses:

$$\sigma'_a = \sigma_a = 157.9\ MPa$$

$$\sigma'_m = \sqrt{3} * \tau_m = 292.5\ MPa$$

Modified Goodman Criterion:

$$\frac{1}{n_f} = \frac{\sigma'_a}{S_e} + \frac{\sigma'_m}{S_u} = 1.634$$

$$n_f = \frac{1}{1.634} = .61$$

Since the fatigue factor of safety is under the requirement of $n_f \geq 2.0$, the stepped shaft fails. This means potential fatigue failure under cyclic conditions, leading to crack initiation and propagation at stress concentration points like fillets, holes or steps.

Finite Element Analysis

A 3D Model was created in SolidWorks Simulation to validate the analytical predictions. The model had a 1 mm fillet radii at the start of the 1st and at the end of the last segment of the stepped shaft. It had the AISI 1020 Cold Drawn material properties (E = 205 GPA, S_y = 390 MPa and S_u = 470 MPa. The mesh was a mix of Coarse and fine and there were bearing supports at the bearing locations (1st and 8th segment).

Key FEA Results:

- Maximum displacement .185 mm
- Maximum Von Mises Stress: 207.7 MPa
- Maximum bending stress:148.4 MPa
- Maximum shear stress (XY plane): 114.2 MPa

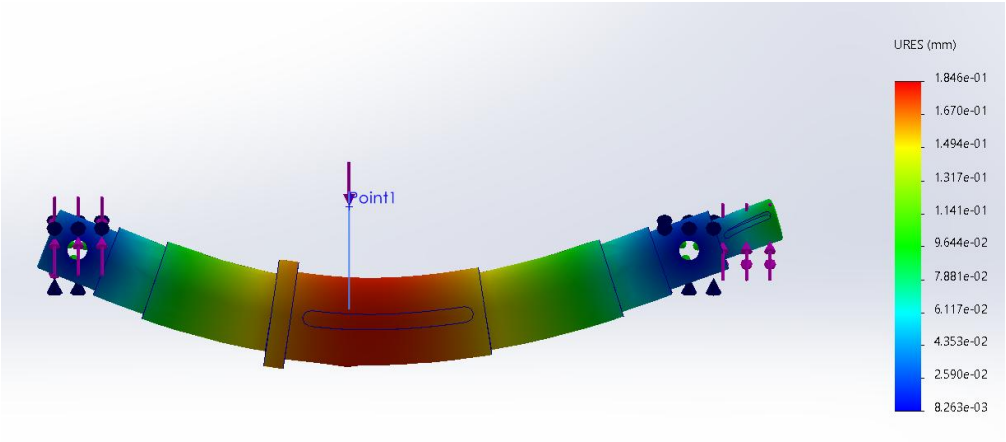


Figure 5: The Displacement Plot from FEA Analysis:

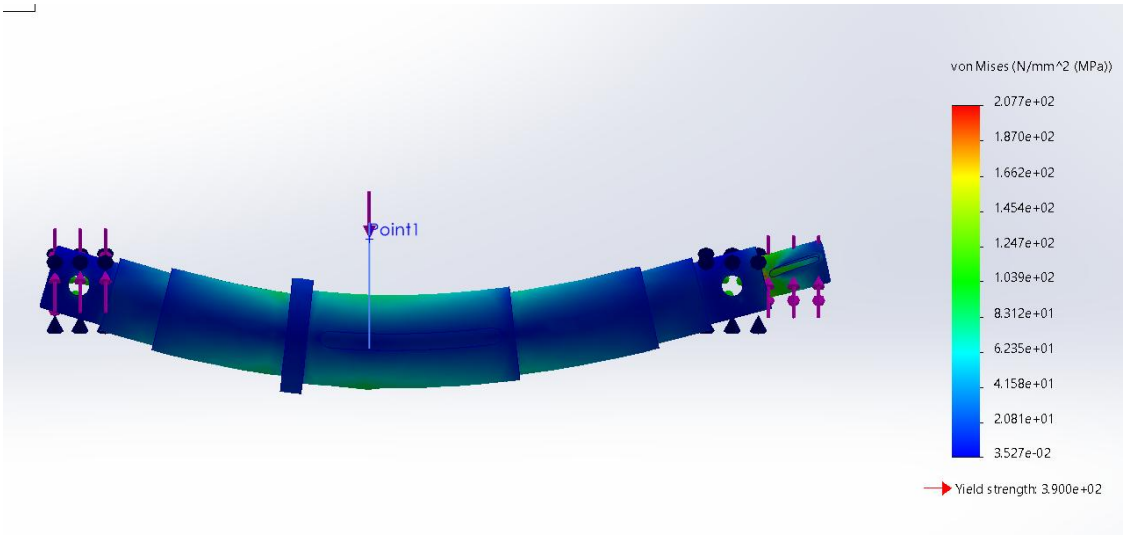


Figure 6: The Von Mises Stress Plot from FEA Analysis:

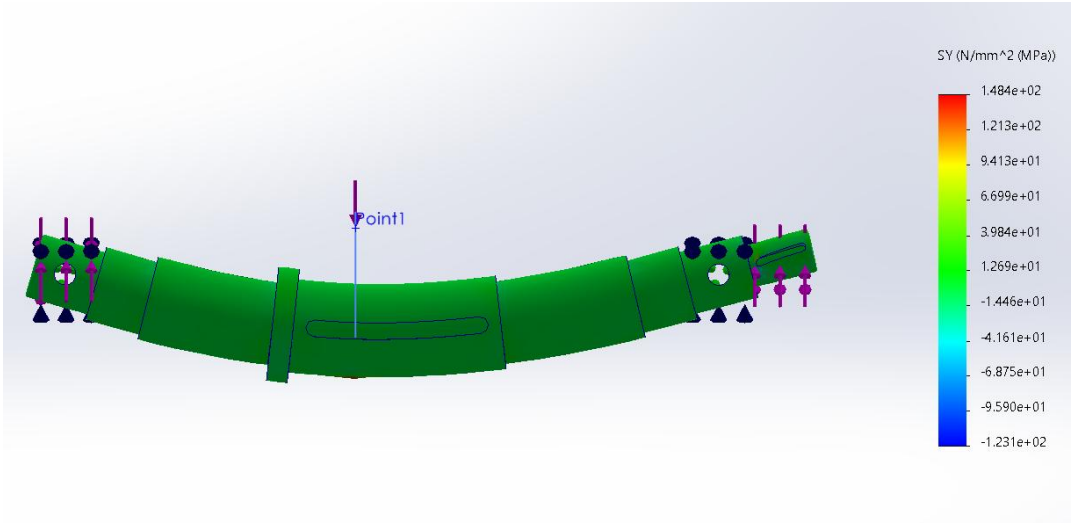


Figure 7:The Normal Stress Plot from FEA Analysis:

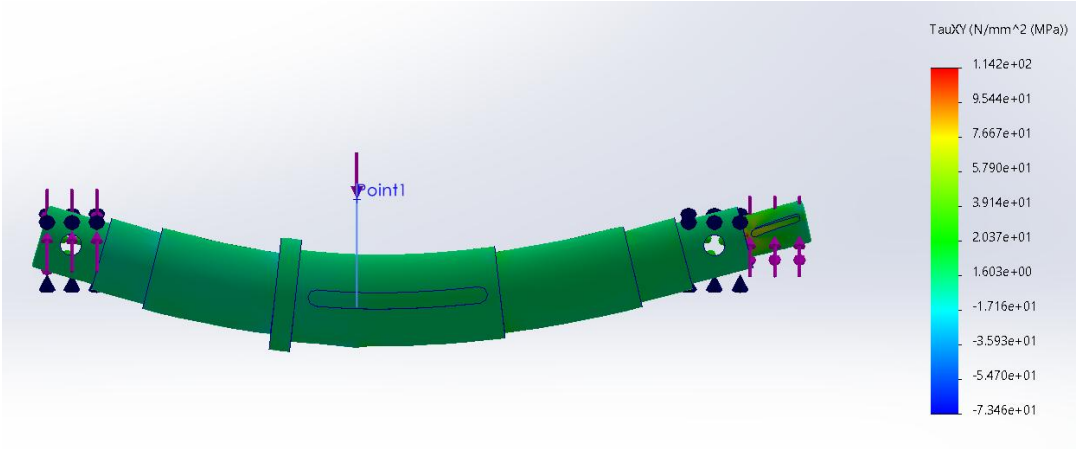


Figure 8: The Shear Stress Plot from FEA Analysis:

With these numbers the static factor of safety (yield) would be:

$$n_y = \frac{390}{207.7} = 1.88$$

Which is above the requirement of $n_y \geq 1.5$, so the part should pass.

While the Static factor of safety for fatigue would be:

$$\frac{1}{n_f} = \frac{\sigma'_a}{S_e} + \frac{\sigma'_m}{S_u} = .897$$

$$n_f = \frac{1}{.897} = 1.12$$

Since the factor of safety is less than the requirement of $n_f \geq 2.0$, the stepped shaft model fails due to fatigue also.

As you can see, there were discrepancies between the numbers from the FEA and the code. This could be from the code not treating stress concentrations and load distribution. Or because of certain geometries that are more visible in the model than in the code. Which could overestimate the stresses. The FEA model accounts for the actual geometry and distributes stress more realistically, resulting in lower peak stresses and deformation. Despite the discrepancies, both analyses the smallest diameter region as the critical location, and that the design is limited.

Sensitivity

Since the load is shifted from it's original position of 155 mm from the left I'd have to calculate the reactions and write the singularity functions again:

$$R_B = \frac{P * a}{L} = \frac{10000 * 165}{375} = 4400\ N$$

$$R_A = P - R_B = 5600\ N$$

Using singularity functions:

$$M(x) = R_A\langle x\rangle^1 - P\langle x - 165\rangle^1$$

The stepped nature make the singularity function for the M/I function formulated again like this:

$$\frac{M}{I} = \frac{R_A}{I_1}\langle x\rangle^1 - \frac{P}{I_k}\langle x - 165\rangle^1 + \sum_{i=1}^{8}\left(\frac{1}{I_{i+1}} - \frac{1}{I_i}\right)[M(x_i)\langle x - x_i\rangle^0 + V(x_i)\langle x - x_i\rangle^1]$$

The updated plots for the load shift would be:

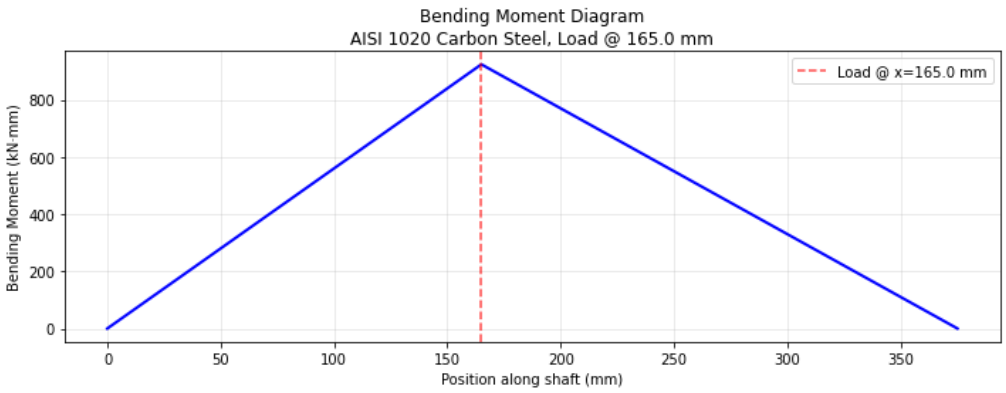


Figure 9: Bending moment diagram after load shift

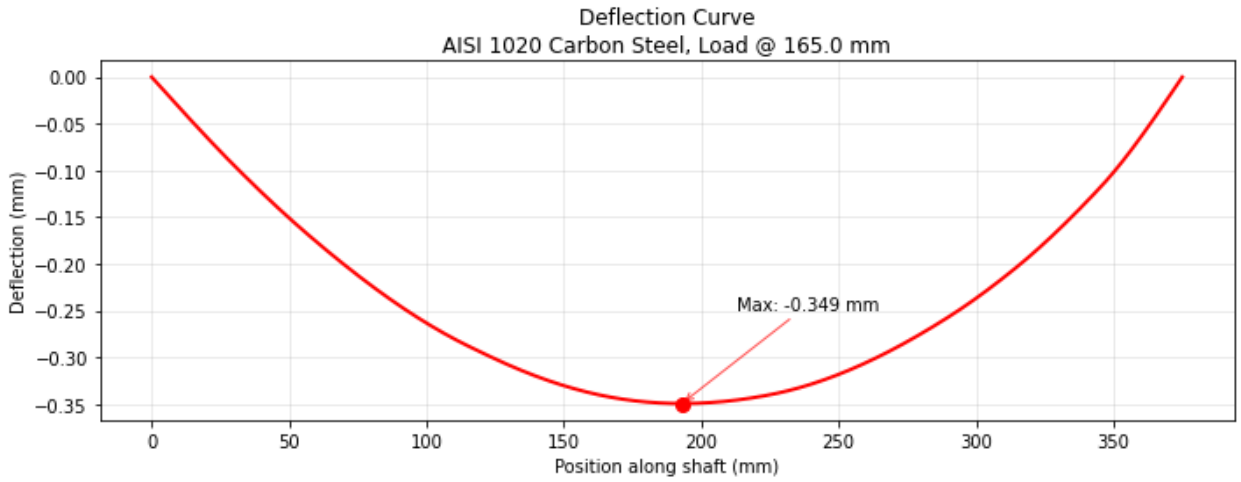


Figure 10: Displacement Diagram after load shift

When the load is shifted 10 mm to the right, the results for everything differ. Here are the stress results from code:

- Maximum displacement .349 mm
- Maximum Von Mises Stress: 457.9 MPa
- Maximum bending stress: 294.1 MPa
- Maximum shear stress (XY plane): 202.6 MPa

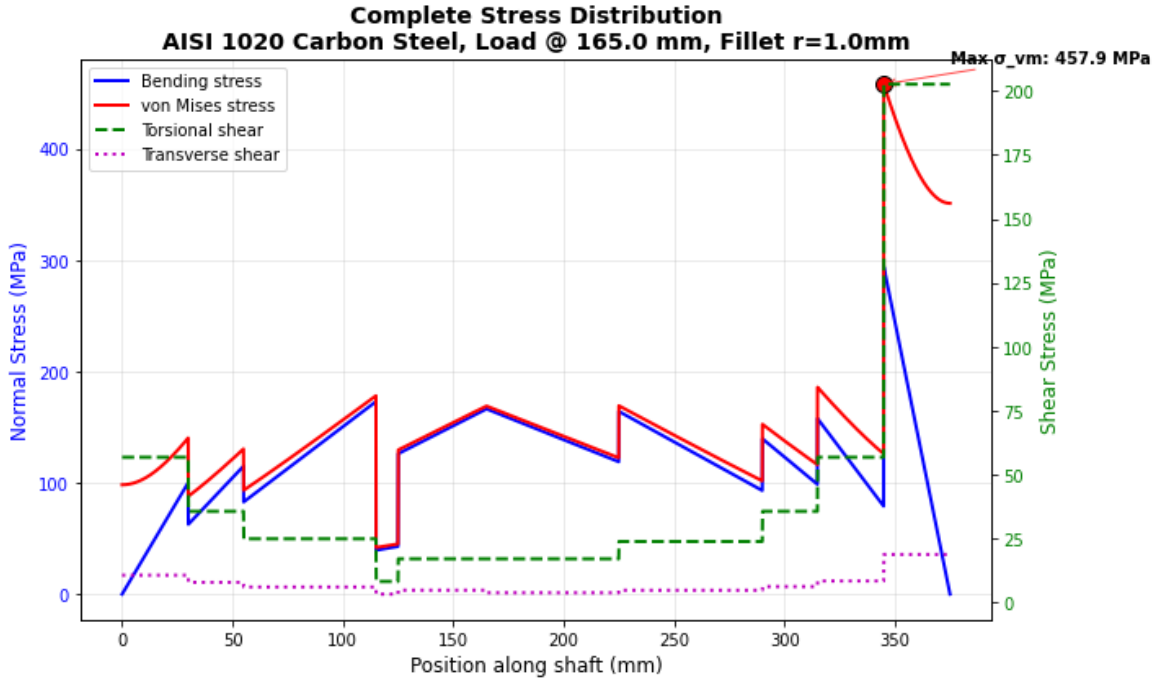


Figure 11: Stress Distribution along the shaft after load shift

In the FEA analysis, the results look completely different as well but there are still discrepancies between the values of the code and the FEA model:

- Maximum displacement .192 mm
- Maximum Von Mises Stress: 205.8 MPa
- Maximum bending stress:121.5 MPa
- Maximum shear stress (XY plane): 113.9 MPa

Static factor of safety (against yielding) results:

Compute $n_y = \frac{S_y}{\sigma_{vm}}$:

Code: $n_y = \frac{S_y}{\sigma_{vm}} = \frac{390}{457.9} = .85 \rightarrow$ Fail (under ≥ 1.5)

FEA: $n_y = \frac{S_y}{\sigma_{vm}} = \frac{390}{205.8} = 1.9 \rightarrow$ Pass.

Fatigue Check (Modified Goodman):

Code: $\frac{1}{n_f} = \frac{\sigma'_a}{S_e} + \frac{\sigma'_m}{S_u} = 1.69 \rightarrow \mathrm{n_f} = \frac{1}{1.69} = .59 \rightarrow$ Fail (under ≥ 2.0)

FEA: $\frac{1}{n_f} = \frac{\sigma'_a}{S_e} + \frac{\sigma'_m}{S_u} = .809 \rightarrow \mathrm{n_f} = \frac{1}{.809} = 1.24 \rightarrow$ Fail (under ≥ 2.0)

In conclusion, when the load is shifted 10 mm to the right, both the code shows a increase in bending and von Mises stress, predicting both static and fatigue failure, while the FEA showed a decrease in stress values. Again, the discrepancies are probably due to the 3D model FEA having more accurate and smoother calculations. While the code doesn't take into effect everything including stress concentrations. Despite this, both show that the shaft fails the fatigue check and that the critical parameter is the minimum shaft diameter and its location relative to the bending moment peak.

Design Tradeoff for Second Material

Calculations for the AISI 4140 Steel Alloy Q&T

Given:

- Material: AISI 4140 Alloy Steel Q&T
- Yield Strength: 600 MPa
- Ultimate Tensile Strength: 690 MPa

Code Analysis

- Maximum displacement .340 mm
- Maximum Von Mises Stress: 345.0 MPa
- Maximum bending stress: 315.8 MPa
- Maximum shear stress (XY plane): 193.6 MPa

Shifted 10 mm right:

- Maximum displacement .349 mm
- Maximum Von Mises Stress: 345 MPa
- Maximum bending stress: 336.1 MPa
- Maximum shear stress (XY plane): 193.6 MPa

FEA Analysis:

- Maximum displacement .184 mm
- Maximum Von Mises Stress: 205.8 MPa
- Maximum bending stress:121.5 MPa
- Maximum shear stress (XY plane): 113.9 MP

Shifted 10 mm right:

- Maximum displacement .189 mm
- Maximum Von Mises Stress: 205 MPa
- Maximum bending stress:121.5 MPa
- Maximum shear stress (XY plane): 115 MPa

Sources of discrepancies could be due to stress concentration treatment, or wrong stress distribution in the code.

Factor of Safety calculations:

Static Factor of Safety (against yielding):

Code Analysis:

- At 155 mm, $n_y = \frac{S_y}{\sigma_{vm}} = 1.74$
- At 165 mm, $n_y = \frac{S_y}{\sigma_{vm}} = 1.74$

FEA Analysis:

- At 155 mm, $n_y = \frac{S_y}{\sigma_{vm}} = 2.915$
- At 165 mm, $n_y = \frac{S_y}{\sigma_{vm}} = 2.927$

Fatigue Factor of Safety:

Code Analysis:

- At 155 mm, $\frac{1}{n_f} = \frac{\sigma'_a}{S_e} + \frac{\sigma'_m}{S_u} = 1.88 \rightarrow n_f = .53$
- At 165 mm, $\frac{1}{n_f} = \frac{\sigma'_a}{S_e} + \frac{\sigma'_m}{S_u} = 1.96 \rightarrow n_f = .51$

FEA Analysis:

- At 155 mm, $\frac{1}{n_f} = \frac{\sigma'_a}{S_e} + \frac{\sigma'_m}{S_u} = 0.8537 \rightarrow n_f = 1.17$
- At 165 mm, $\frac{1}{n_f} = \frac{\sigma'_a}{S_e} + \frac{\sigma'_m}{S_u} = 0.8565 \rightarrow n_f = 1.17$

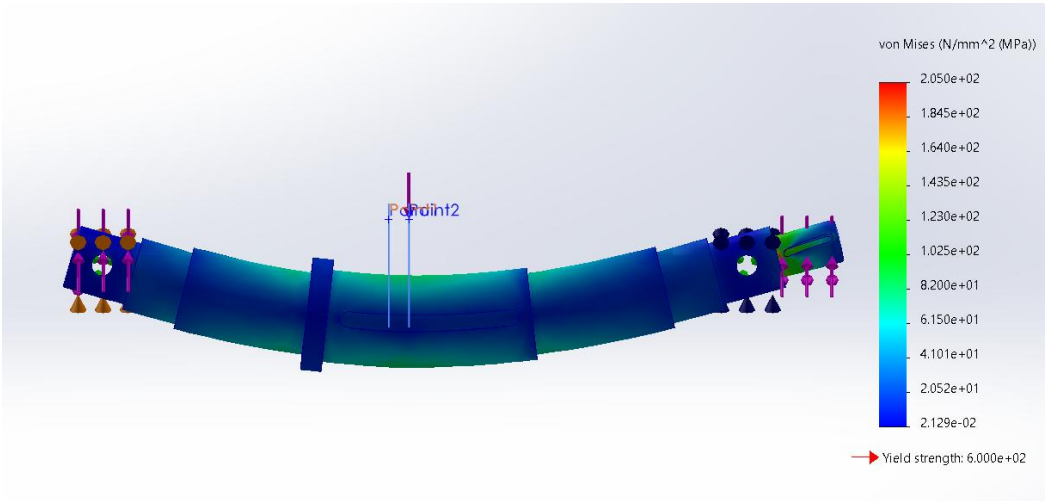


Figure 12: Von Mises Plot for the 2nd Material

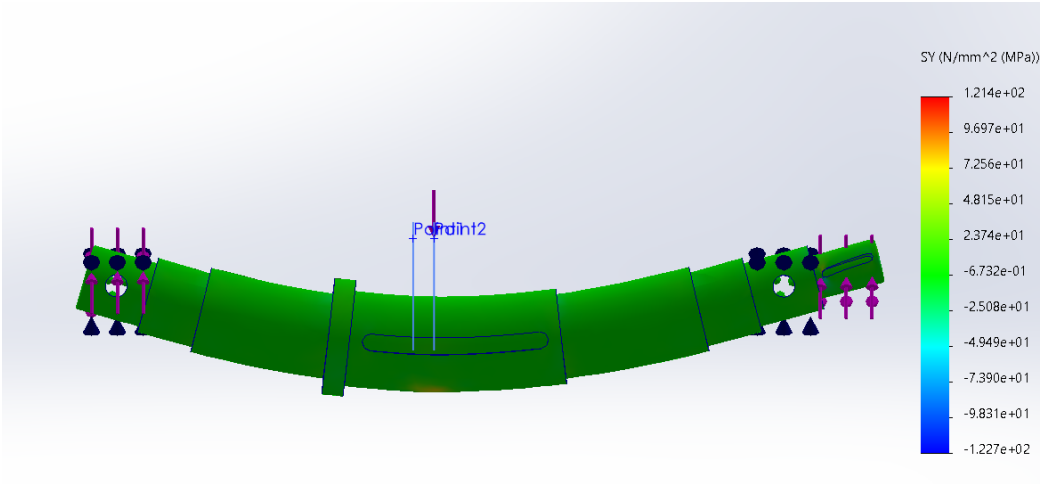


Figure 13: Normal Stress Plot for the 2nd Material

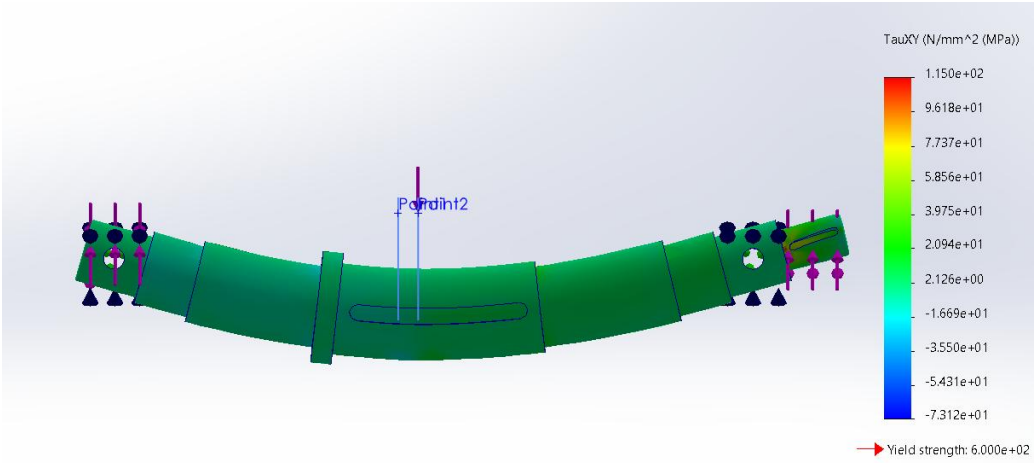


Figure 14: Shear Stress Plot for the 2nd Material

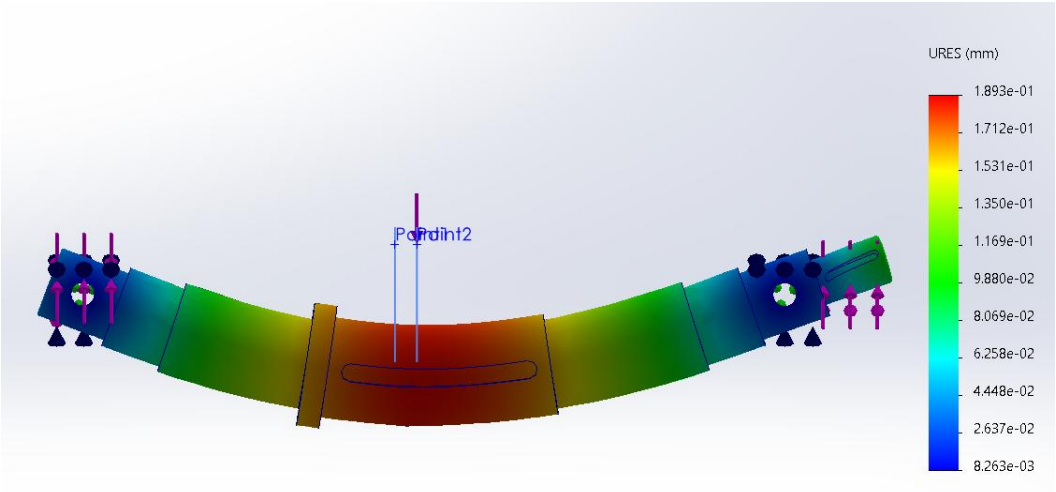


Figure 15: Displacement Plot for the 2nd Material

Observations and Comparison:

1. The FEA analysis predicts significantly lower maximum displacement compared to the code for same loading conditions. The Load shift of 10 mm increases the deflection slightly though in both analyses, but the change is fairly small, indicating the geometry is relatively stiff and not sensitive to load position.
2. Maximum von mises stress, bending stress and shear stress are consistently lower in the FEA model than in the code analysis. Also, since the geometry didn't change as much, there was no change in most of the stress values. This difference is likely because of simplified assumptions in the code, stress concentration approximation and uniform stress distribution. The load shift affects the bending stress but has no effect on FEA stress, meaning that the FEA captures real stress distribution more accurately.
3. Using the yield strength $S_y = 600\text{MPa}$, the static factor of safety for the code is approximately 1.74 and for the FEA model, it is approximately 2.92 regardless of the load shift. The higher FEA Factor of safety is consistent with lower von Mises stresses predicted by the model.
4. For the Fatigue factor of safety, the code predicts low factor of safety around 0.52 indicating failure under cyclic loading. While the FEA analysis predicts a higher fatigue Factor of safety around 1.17, which is still under the $n_f \geq 2.0$ requirement.
5. Shifting the load slightly changes the stresses and factors of safety in both the code and FEA model. The FEA model shows minimal change, meaning the design is relatively sturdy and unresponsive to load position. The code is more sensitive probably because of simplified assumptions.

Conclusion:

Comparing the two shaft materials shows that, although their stiffness results are similar due to identical geometry, their strength performance differs significantly. The original AISI 1020 cold-drawn shaft consistently failed both the static and fatigue criteria in the analytical model, primarily because the minimum 20 mm diameter segment experiences high combined bending and torsional stresses. The FEA results predicted lower stress than the code but still confirmed that the shaft would fail due to fatigue failure as the governing mode.

In contrast, the second material (with higher yield and ultimate strength) improved the static factor of safety in both the code and the FEA models and was over the requirement of $n_y \geq 1.5$, indicating the shaft wouldn't succumb to yielding. However, even with stronger material, the fatigue factor of safety remained below the required $n_f \geq 2.0$, indicating that the design is ultimately fatigue limited rather than strength limited.

Both analyses agree that the critical parameter is the minimum shaft diameter, since it controls stiffness, stress concentration severity, and fatigue resistance. Improving the design therefore requires either increasing this diameter, reshaping stress concentrated transitions, or reducing

the applied torque and bending loads. Despite the discrepancies between the results of the code and the FEA model, both consistently identify the same failure mechanism and the same geometric weakness. This shows that changing the material alone isn’t going to make the design safe under the required fatigue life.

Design Justification and Recommendation:

As I said in the last section, changing the material alone isn't enough for the shaft to meet both static and fatigue factors of safety requirements. This essentially means that geometric modifications are required. If we had to use one of the materials out of the two it would have to be AISI 4140 Alloy Steel. Since it fulfills more requirements than the AISI 1020 cold drawn carbon steel.

Through iterative testing with the code, if you could change the material, I found that using Beryllium and changing the diameters of the steps fulfills every requirement. Using Beryllium helps the shaft fulfill the slope requirement since it has a higher elastic modulus than AISI 4140 Alloy Steel and the increase in diameters increases the inertia at the load location it causes the slope of the deflection to be lower. Doing this also increases the fatigue factor of safety and the static factor of safety. Although it does fulfill all the requirements the material is expensive and could be hard to find.

If you had to make this shaft like this, you would have to change the diameter sizes for sure but also find a material that has a Elastic modulus over 300 GPa for it to fulfill the bearing slope factor of safety requirements. I also thought that coating the AISI 4140 Alloy Steel with carbon fiber could increase the elastic modulus a lot and it is relatively less expensive than using other materials. This also allows the material to have around the same mass since carbon is incredibly lightweight and can handle cyclic loading.

Proposed Changes

In all, the most practical solution would be a moderate diameter increase combined with better fillet radii. It addresses both stress and stiffness concerns and causes no increase in cost and complexity. All I did was increase the diameters by 10 mm and changed the material to AISI 4140 Alloy Steel and it passed all the requirements. The static FOS and Fatigue FOS are 5.96 and 3.33 which are above both the requirements of 1.5 and 2. The slope of the bearings and gears are both under .001 and .0005. The mass is 6.55 kg. The Max von Mises Stress is 109.1 MPa, bending stress is 63.8 MPa, and max shear stress is 54.8 MPa. The max deflection is .080 mm.

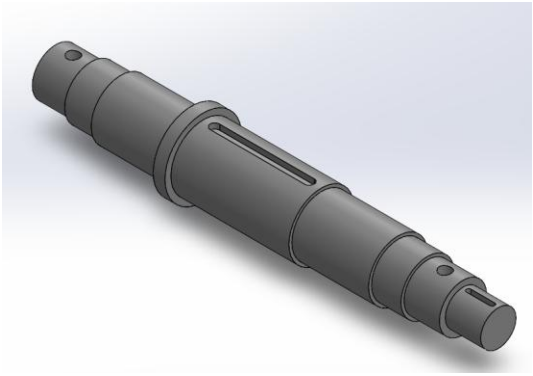


Figure 16: Updated Design

Design Standard

A relevant design standard for rotating shafts is ASME B106.1M, “Design of Transmission Shafting.” This standard provides guidelines for allowable stress, fatigue considerations, stress concentration factors, and recommended safety factors for the shafts subjected to combined bending and torsion. In this project this design standard can be used to verify that the selected material and geometry meet recommended limits for alternating and mean stress, ensure that stress concentration factors are applied correctly, and confirm that the chosen safety factors align with accepted industry practice for rotating machinery. Incorporating ASME B106.1M ensures the final shaft design is consistent with established engineering practice and provides sufficient margin for long-term reliability.

Professional Failure Memo:

To: Engineering firm

From: Aaradhaya Sutrave

Date: 12/8/2025

Subject: Engineering Hire, Lessons from the Aloha Airlines 243

On April 28, 1988, Aloha Airlines Flight 243, a Boeing 737 200, experienced catastrophic structural failure at 24000 ft when an 18 foot section of the airplane's upper fuselage tore away, causing a explosive decompression. One flight attendant was lost, and many passengers were injured, though the crew managed a successful emergency landing. Investigations by the National Transportation Safety Board revealed that the root cause was extensive fatigue cracking along a fuselage joint. Over years of pressurization cycles, moisture breach and corrosion caused small cracks to form at the rivet holes and disbanded regions. These micro cracks joined together and caused a sudden failure. The original lap-joint design, maintenance inspection methods and lack of damage tolerance criteria were all contributing factors.

This failure could have been predicted and prevented using the same analysis techniques we applied in Part II of our project:

- Stress-concentration factors (K_T): The rivet holes, lap-joint geometries, and disbonded regions acts as geometric holes or cracks, significantly increasing local stress beyond nominal values. If we had calculated K_T and adjusted stress accordingly, the local stress near rivets would have shown dangerous peaks.
- Alternating and mean stress analysis: Every pressurization cycle generated an alternating stress and spread out causing mean hoop stress. By calculating the alternating stress amplitude and mean stress, a fatigue life analysis could have revealed that the joint was operating in an unsafe region at least.
- Goodman criterion: plotting the stress state on a Goodman diagram would likely have shown that the fatigue factor of safety was below the acceptable thresholds of the factor of safety listed in the design standard related to fuselage or aerospace walls. This would have flagged the design for either revision, increased inspection, or repairs.
- Damage tolerance philosophy: Instead of assuming no flaws, a design based on damage tolerance would have anticipated crack initiation at stress concentrators and required use of sturdier joint geometry

If these methods were used during design or prior to the pressurization cycles, this incident could have been avoided long before the catastrophic failure.

Modern engineering employers value skills needed to prevent failures like Flight 243. For example, a Structures Analyst / Design Engineer would be doing work related to FEA Analysis, fatigue analysis, damage and tolerance assessments, stress concentration analysis, and structural integrity evaluations. Here's one that Boeing posted few days ago:



As an engineer mastering "FEA analysis, fatigue life calculations, and understanding geometric stress increases" helps to design components that remain safe over many cycles, help predict where cracks may form, justify geometry changes or maintenance scheduling, and help prevent catastrophic failures that can put lives at risk.

The Aloha Flight 243 incident shows that skipping these practices is not an option, since if done again, it can cause another catastrophic accident to occur. Cultivating these skills can help a student create safer, more reliable, and industry compliant designs.

References:

Federal Aviation Administration. “Boeing 737-200 | Federal Aviation Administration.” *Www.faa.gov*, 19 Dec. 2022, www.faa.gov/lessons_learned/transport_airplane/accidents/N73711.

Kanghui Song, Dawei Li, Tingting Liu, Changdong Zhang, Yi Min Xie, Wenhe Liao,

Crystal-twinning inspired lattice metamaterial for high stiffness, strength, and toughness,

Materials & Design,

Volume 221,

2022,

110916,

ISSN 0264-1275,

https://doi.org/10.1016/j.matdes.2022.110916.

(https://www.sciencedirect.com/science/article/pii/S026412752200538X)

Yue Zhang, Xian Wang, Ruitao Peng, Changyou Xu, Tao Shi, Kexi Chen,

Stiffness degradation failure mechanisms and probabilistic fatigue life analysis of TA1 clinched joints,

Engineering Failure Analysis,

Volume 179,

2025,

109812,

ISSN 1350-6307,

https://doi.org/10.1016/j.engfailanal.2025.109812.

(https://www.sciencedirect.com/science/article/pii/S1350630725005539)

Appendix:

Code used to solve the problem analytically:

```
# -*- coding: utf-8 -*-

"""

Created on Mon Dec  8 22:28:18 2025


@author: aarad

"""



import numpy as np

import math

import matplotlib.pyplot as plt

from pathlib import Path


def cumtrapz_custom(y, x):

    """Cumulative trapezoidal integration with initial=0"""

    y = np.asarray(y)

    x = np.asarray(x)

    n = len(x)

    res = np.zeros(n)

    for i in range(1, n):

        res[i] = res[i-1] + 0.5 * (y[i] + y[i-1]) * (x[i] - x[i-1])

    return res


def calculate_stress_concentration(D, d, r):

    """Calculate stress concentration factors for stepped shaft with fillet"""

    if D <= d or r <= 0:

        return 1.0, 1.0


    # Simpler, more realistic formulas

    # Kt typically ranges from 1.2 to 2.0 for reasonable fillets

    ratio = D/d
```

```
    # For bending - Peterson's approximate formula
    if r/d < 0.1:
        Kt_bending = 1.5 + 0.5 * (ratio - 1)
    else:
        Kt_bending = 1.2 + 0.3 * (ratio - 1)

    # For torsion - typically lower than bending
    Kt_torsion = 1.1 + 0.2 * (ratio - 1)

    # Reasonable limits
    return min(Kt_bending, 2.0), min(Kt_torsion, 1.5)

def A_from_d(d):
    """Calculate cross-sectional area"""
    return math.pi * d**2 / 4.0

def calculate_max_transverse_shear(V, d):
    """Calculate maximum transverse shear stress (at neutral axis)"""
    A = A_from_d(d)
    return (4/3) * abs(V) / A if A > 0 else 0

# ====================
# USER INPUT SECTION
# ====================

print("="*60)
print("STEPPED SHAFT DESIGN ANALYSIS (REALISTIC VALUES)")
print("="*60)

# Material selection
print("\nMATERIAL SELECTION:")
print("  1: AISI 1020 cold-drawn Carbon Steel")
print("     S_u = 470 MPa, S_y = 390 MPa")
print("  2: AISI 4140 Q&T Alloy Steel")
print("     S_u = 690 MPa, S_y = 600 MPa")
```

```
while True:
    try:
        material_choice = int(input("\nEnter material choice (1 or 2): "))
        if material_choice in [1, 2]:
            break
        else:
            print("Invalid choice! Please enter 1 or 2.")
    except ValueError:
        print("Invalid input! Please enter 1 or 2.")

# Load position input
print("\nLOAD POSITION:")
print(f"  Total shaft length: 375 mm")
print("  Default load position: 155 mm from left end")

while True:
    try:
        load_position = float(input("\nEnter load position from left end (mm): "))
        if 0 <= load_position <= 375:
            break
        else:
            print(f"Invalid position! Must be between 0 and 375 mm.")
    except ValueError:
        print("Invalid input! Please enter a number.")

# Ask for gear position
while True:
    try:
        gear_position = float(input("Enter gear position for slope check (mm, default 175): ") or "175")
        if 0 <= gear_position <= 375:
            break
        else:
            print(f"Invalid position! Must be between 0 and 375 mm.")
    except ValueError:
```

```
        print("Invalid input! Please enter a number.")

# Ask for fillet radius
print("\nSTRESS CONCENTRATION CONSIDERATION:")
print("  Note: Fillets reduce stress concentration at diameter transitions")
while True:
    try:
        fillet_radius = float(input("Enter fillet radius at steps (mm, recommended 2-5): ") or "2")
        if fillet_radius > 0:
            break
        else:
            print("Fillet radius must be positive!")
    except ValueError:
        print("Invalid input! Please enter a number.")

# ====================
# MATERIAL PROPERTIES
# ====================

if material_choice == 1:
    material_name = "AISI 1020 Carbon Steel"
    S_u = 470.0  # MPa
    S_y = 390.0  # MPa
    k_a = 0.9
else:
    material_name = "AISI 4140 Alloy Steel"
    S_u = 690.0  # MPa
    S_y = 600.0  # MPa
    k_a = 0.9

# Common material properties
E = 205000.0  # MPa
rho = 7850.0  # kg/m³

# Marin factors
```

```
k_b = 0.85
k_c = 1.0
k_d = 1.0
k_e = 0.868
k_f = 1.0

# ====================
# LOADING CONDITIONS
# ====================

P = 10000.0  # N
a_load = load_position
L = 375.0
gear_x = gear_position

# Power transmission - FIXED CALCULATION
P_power = 25000.0  # W
n_rpm = 900.0
omega = 2 * math.pi * n_rpm / 60.0
T = (P_power / omega)  # N·m (not N·mm!)

print(f"\nTorque calculation:")
print(f"  Power: {P_power} W")
print(f"  Speed: {n_rpm} RPM")
print(f"  Angular velocity: ω = {omega:.2f} rad/s")
print(f"  Torque: T = P/ω = {P_power}/{omega:.2f} = {T:.2f} N·m = {T*1000:.0f} N·mm")

T = T * 1000  # Convert to N·mm for calculations

# ====================
# SHAFT GEOMETRY
# ====================

segment_lengths = np.array([30, 25, 60, 10, 100, 65, 25, 30, 30], dtype=float)
segment_diams = np.array([30, 35, 40, 55, 45, 40, 35, 30, 20], dtype=float)
```

```
# Segment boundaries
x_ends = np.cumsum(segment_lengths)
x_starts = np.concatenate(([0.0], x_ends[:-1]))

def calculate_mass(diameters, lengths, density=rho):
    """Calculate shaft mass in kg"""
    volume = 0
    for d, l in zip(diameters, lengths):
        area = math.pi * (d/1000)**2 / 4
        volume += area * (l/1000)
    return volume * density

mass = calculate_mass(segment_diams, segment_lengths)

# ====================
# ANALYTICAL FUNCTIONS
# ====================

def I_from_d(d):
    return math.pi * d**4 / 64.0

def J_from_d(d):
    return math.pi * d**4 / 32.0

I_segments = np.array([I_from_d(d) for d in segment_diams])
J_segments = np.array([J_from_d(d) for d in segment_diams])
c_segments = segment_diams / 2.0

def ustep(val):
    return np.heaviside(val, 1.0)

def local_segment_index(x_val):
    """Return segment index for a given x position"""
    if x_val >= L:
```

```
        return len(segment_lengths) - 1
    for i, end in enumerate(x_ends):
        if x_val < end:
            return i
    return len(segment_lengths) - 1

def get_stress_concentration_factors(diameters, fillet_radius):
    """Calculate Kt and Kts for each step"""
    Kt_bending = np.ones(len(diameters))
    Kt_torsion = np.ones(len(diameters))

    for i in range(len(diameters)-1):
        if diameters[i+1] > diameters[i]:
            D, d = diameters[i+1], diameters[i]
        else:
            D, d = diameters[i], diameters[i+1]

        Kt_b, Kt_t = calculate_stress_concentration(D, d, fillet_radius)
        # Apply to the smaller diameter (more critical)
        if diameters[i+1] < diameters[i]:
            Kt_bending[i+1] = Kt_b
            Kt_torsion[i+1] = Kt_t
        else:
            Kt_bending[i] = Kt_b
            Kt_torsion[i] = Kt_t

    return Kt_bending, Kt_torsion

# ====================
# MAIN ANALYSIS FUNCTION
# ====================

def analyze_shaft(material_name, S_u, S_y, a_load, gear_x):
    """Analyze stepped shaft with given parameters"""
```

```
Kt_bending, Kt_torsion = get_stress_concentration_factors(segment_diams, fillet_radius)


R_B = P * a_load / L

R_A = P - R_B


print(f"\nReaction forces:")

print(f"  R_A = {R_A:.1f} N")

print(f"  R_B = {R_B:.1f} N")


x = np.linspace(0.0, L, 5001)


def M_of_x(x_val):

    return R_A * x_val - P * max(0.0, x_val - a_load)


def V_of_x(x_val):

    return R_A - P * (1.0 if x_val > a_load else 0.0)


M = np.vectorize(M_of_x)(x)

V = np.vectorize(V_of_x)(x)


step_locations = x_ends[:-1]

Ma = np.array([M_of_x(a) for a in step_locations])

Va = np.array([V_of_x(a) for a in step_locations])


M_over_I_explicit = np.zeros_like(x)

M_over_I_explicit += (R_A / I_segments[0]) * ustep(x - 0.0) * (x - 0.0)


idx_load_seg = local_segment_index(a_load)

I_load = I_segments[idx_load_seg]

M_over_I_explicit -= (P / I_load) * ustep(x - a_load) * (x - a_load)


for i, a in enumerate(step_locations):

    invI_diff = (1.0 / I_segments[i+1] - 1.0 / I_segments[i])

    term = invI_diff * (Ma[i] * ustep(x - a) * (x - a)**0 + Va[i] * ustep(x - a) * (x - a)**1)

    M_over_I_explicit += term
```

```
M_over_EI = M_over_I_explicit / E
theta_raw = cumtrapz_custom(M_over_EI, x)
y_raw = cumtrapz_custom(theta_raw, x)

C1 = -y_raw[-1] / L
theta = theta_raw + C1
y = y_raw + C1 * x

max_deflection = np.min(y)
max_deflection_x = x[np.argmin(y)]

theta_left = theta[0]
theta_right = theta[-1]
gear_idx = np.argmin(np.abs(x - gear_x))
theta_gear = theta[gear_idx]

# Stress analysis - FIXED CALCULATIONS
sigma_b_surface = np.zeros_like(x)
tau_t_surface = np.zeros_like(x)
tau_s_NA = np.zeros_like(x)
sigma_vm_surface = np.zeros_like(x)

# For debugging
sigma_b_nominal = np.zeros_like(x)
tau_t_nominal = np.zeros_like(x)

for i, xv in enumerate(x):
    seg = local_segment_index(xv)
    I_loc = I_segments[seg]
    c_loc = c_segments[seg]
    J_loc = J_segments[seg]
    d_loc = segment_diams[seg]

    Mb = M[i]
```

```
    V_loc = V[i]

    # Calculate nominal stresses
    sigma_b_nominal[i] = (Mb * c_loc / I_loc) if I_loc > 0 else 0
    tau_t_nominal[i] = (T * c_loc / J_loc) if J_loc > 0 else 0

    # Apply stress concentration factors
    sigma_b_surface[i] = Kt_bending[seg] * sigma_b_nominal[i]
    tau_t_surface[i] = Kt_torsion[seg] * tau_t_nominal[i]

    # Transverse shear (at neutral axis, not at surface)
    tau_s_NA[i] = calculate_max_transverse_shear(V_loc, d_loc)

    # Von Mises at surface (bending + torsion only - transverse shear is 0 at surface)
    sigma_vm_surface[i] = np.sqrt(sigma_b_surface[i]**2 + 3.0 * tau_t_surface[i]**2)

crit_idx = np.argmax(sigma_vm_surface)
crit_x = x[crit_idx]
crit_seg = local_segment_index(crit_x)
crit_d = segment_diams[crit_seg]
crit_sigma_vm = sigma_vm_surface[crit_idx]
crit_sigma_b = sigma_b_surface[crit_idx]
crit_tau_t = tau_t_surface[crit_idx]

# Store nominal values at critical point
crit_sigma_b_nominal = sigma_b_nominal[crit_idx]
crit_tau_t_nominal = tau_t_nominal[crit_idx]

max_tau_t = np.max(tau_t_surface)
max_tau_t_x = x[np.argmax(tau_t_surface)]
max_tau_s = np.max(tau_s_NA)
max_tau_s_x = x[np.argmax(tau_s_NA)]

# Fatigue stress components
mean_bending_stress = 0.0  # Rotating shaft
```

```
alternating_bending_stress = np.max(np.abs(sigma_b_surface))
mean_torsional_stress = crit_tau_t  # Steady torque

# Stiffness constraints
theta_bearing_max = 0.001
theta_gear_max = 0.0005

FoS_bearing = theta_bearing_max / abs(theta_right) if abs(theta_right) > 0 else 999
FoS_gear = theta_gear_max / abs(theta_gear) if abs(theta_gear) > 0 else 999

# Static FoS
n_y = S_y / crit_sigma_vm if crit_sigma_vm > 0 else 999

# Fatigue analysis
S_e_prime = 0.5 * S_u
S_e = S_e_prime * k_a * k_b * k_c * k_d * k_e * k_f

# Notch sensitivity (simplified)
q = 0.9  # For steel with moderate fillets

Kf_bending = 1 + q * (Kt_bending[crit_seg] - 1)
Kf_torsion = 1 + q * (Kt_torsion[crit_seg] - 1)

sigma_a = Kf_bending * alternating_bending_stress
tau_m = Kf_torsion * mean_torsional_stress

sigma_a_prime = sigma_a  # Alternating component (bending fully reversed)
sigma_m_prime = np.sqrt(3) * tau_m  # Mean component (steady torsion)

if sigma_a_prime > 0 or sigma_m_prime > 0:
    n_f = 1 / (sigma_a_prime/S_e + sigma_m_prime/S_u)
else:
    n_f = 999

print(f"\nCritical point analysis (x = {crit_x:.1f} mm, d = {crit_d} mm):")
```

```
print(f"  Nominal bending stress: {crit_sigma_b_nominal:.1f} MPa")
print(f"  Nominal torsional stress: {crit_tau_t_nominal:.1f} MPa")
print(f"  Kt_bending: {Kt_bending[crit_seg]:.2f}")
print(f"  Kt_torsion: {Kt_torsion[crit_seg]:.2f}")
print(f"  Actual bending stress (with Kt): {crit_sigma_b:.1f} MPa")
print(f"  Actual torsional stress (with Kts): {crit_tau_t:.1f} MPa")
print(f"  Von Mises stress: {crit_sigma_vm:.1f} MPa")

return {
    "material": material_name,
    "load_position": a_load,
    "gear_position": gear_x,
    "fillet_radius": fillet_radius,
    "R_A": R_A,
    "R_B": R_B,
    "torque": T,
    "mass": mass,
    "max_deflection": max_deflection,
    "max_deflection_x": max_deflection_x,
    "theta_left": theta_left,
    "theta_right": theta_right,
    "theta_gear": theta_gear,
    "FoS_bearing": FoS_bearing,
    "FoS_gear": FoS_gear,
    "crit_x": crit_x,
    "crit_d": crit_d,
    "crit_sigma_b": crit_sigma_b,
    "crit_sigma_b_nominal": crit_sigma_b_nominal,
    "crit_tau_t": crit_tau_t,
    "crit_tau_t_nominal": crit_tau_t_nominal,
    "crit_sigma_vm": crit_sigma_vm,
    "max_tau_t": max_tau_t,
    "max_tau_t_x": max_tau_t_x,
    "max_tau_s": max_tau_s,
    "max_tau_s_x": max_tau_s_x,
```

```
        "mean_bending_stress": mean_bending_stress,
        "alternating_bending_stress": alternating_bending_stress,
        "mean_torsional_stress": mean_torsional_stress,
        "n_y": n_y,
        "n_f": n_f,
        "S_e": S_e,
        "S_u": S_u,
        "Kt_bending": Kt_bending[crit_seg],
        "Kt_torsion": Kt_torsion[crit_seg],
        "x": x,
        "M": M,
        "V": V,
        "M_over_I": M_over_I_explicit,
        "y": y,
        "theta": theta,
        "sigma_b_surface": sigma_b_surface,
        "tau_t_surface": tau_t_surface,
        "tau_s_NA": tau_s_NA,
        "sigma_vm_surface": sigma_vm_surface,
        "sigma_b_nominal": sigma_b_nominal,
        "tau_t_nominal": tau_t_nominal
    }

# ====================
# RUN ANALYSIS
# ====================

results = analyze_shaft(material_name, S_u, S_y, a_load, gear_x)

# ====================
# PRINT RESULTS
# ====================

print("\n" + "="*60)
print("ANALYSIS RESULTS (WITH REALISTIC STRESSES)")
```

```
print("="*60)


print(f"\nLOADING:")
print(f"  Reactions: R_A = {results['R_A']:.1f} N, R_B = {results['R_B']:.1f} N")
print(f"  Torque: T = {results['torque']:.0f} N·mm ({results['torque']/1000:.1f} N·m)")
print(f"  Shaft mass: {results['mass']:.3f} kg")

print(f"\nSTRESS CONCENTRATION:")
print(f"  Fillet radius: {fillet_radius} mm")
print(f"  Kt_bending at critical point: {results['Kt_bending']:.2f}")
print(f"  Kt_torsion at critical point: {results['Kt_torsion']:.2f}")

print(f"\nNOMINAL STRESSES AT CRITICAL POINT (without stress concentration):")
print(f"  Bending stress: {results['crit_sigma_b_nominal']:.1f} MPa")
print(f"  Torsional stress: {results['crit_tau_t_nominal']:.1f} MPa")

print(f"\nACTUAL STRESSES AT CRITICAL POINT (with stress concentration):")
print(f"  Location: x = {results['crit_x']:.1f} mm, d = {results['crit_d']:.1f} mm")
print(f"  Bending stress: {results['crit_sigma_b']:.1f} MPa")
print(f"  Torsional stress: {results['crit_tau_t']:.1f} MPa")
print(f"  Von Mises stress: {results['crit_sigma_vm']:.1f} MPa")

print(f"\nMAXIMUM SHEAR STRESSES:")
print(f"  Max torsional shear: {results['max_tau_t']:.1f} MPa at x = {results['max_tau_t_x']:.1f} mm")
print(f"  Max transverse shear: {results['max_tau_s']:.1f} MPa at x = {results['max_tau_s_x']:.1f} mm")

print(f"\nSTIFFNESS ANALYSIS:")
print(f"  Max deflection: {results['max_deflection']:.3f} mm at x = {results['max_deflection_x']:.1f} mm")
print(f"  Slope at left bearing:  {results['theta_left']:.6f} rad")
print(f"  Slope at right bearing: {results['theta_right']:.6f} rad")
print(f"  Slope at gear (x={gear_x} mm): {results['theta_gear']:.6f} rad")

bearing_status = " PASS" if results['FoS_bearing'] >= 1.0 else "✗ FAIL"
gear_status = " PASS" if results['FoS_gear'] >= 1.0 else "✗ FAIL"
print(f"\n  Slope Factors of Safety:")
```

```
print(f"    Bearing: {results['FoS_bearing']:.3f} {bearing_status} (required ≥ 1.0)")
print(f"    Gear:    {results['FoS_gear']:.3f} {gear_status} (required ≥ 1.0)")

print(f"\nFATIGUE STRESS COMPONENTS:")
print(f"  Mean bending stress: {results['mean_bending_stress']:.1f} MPa (rotating shaft)")
print(f"  Alternating bending stress: {results['alternating_bending_stress']:.1f} MPa")
print(f"  Mean torsional stress: {results['mean_torsional_stress']:.1f} MPa (steady torque)")

yield_status = "✓ PASS" if results['n_y'] >= 1.5 else "✗ FAIL"
fatigue_status = "✓ PASS" if results['n_f'] >= 2.0 else "✗ FAIL"

print(f"\nSAFETY FACTORS:")
print(f"  Static FoS (n_y): {results['n_y']:.2f} {yield_status} (required ≥ 1.5)")
if results['n_f'] < 2.0:
    print(f"  Fatigue FoS (n_f): {results['n_f']:.2f} ✗ FAIL")
    print(f"      FATIGUE FAILURE RISK: n_f = {results['n_f']:.2f} < 2.0")
    print(f"    Consider increasing diameters, especially at the {results['crit_d']} mm segment")
else:
    print(f"  Fatigue FoS (n_f): {results['n_f']:.2f} {fatigue_status} (required ≥ 2.0)")

print(f"\nFATIGUE PARAMETERS:")
print(f"  Modified endurance limit (S_e): {results['S_e']:.1f} MPa")
print(f"  Ultimate strength (S_u): {results['S_u']:.1f} MPa")

requirements_met = all([
    results['FoS_bearing'] >= 1.0,
    results['FoS_gear'] >= 1.0,
    results['n_y'] >= 1.5,
    results['n_f'] >= 2.0
])

print("\n" + "="*60)
print("DESIGN REQUIREMENTS CHECK")
print("="*60)
```

```
if requirements_met:
    print("✓ ALL REQUIREMENTS MET")
else:
    print("✗ SOME REQUIREMENTS NOT MET")
    if results['FoS_bearing'] < 1.0:
        print(f"  - Bearing slope exceeds limit")
    if results['FoS_gear'] < 1.0:
        print(f"  - Gear slope exceeds limit")
    if results['n_y'] < 1.5:
        print(f"  - Static FoS too low (n_y = {results['n_y']:.2f})")
    if results['n_f'] < 2.0:
        print(f"  - Fatigue FoS too low (n_f = {results['n_f']:.2f})")

# ====================
# CREATE PLOTS
# ====================

out_dir = Path.cwd() / f"shaft_analysis_realistic_{material_name.replace(' ', '_')}_load{int(a_load)}"
out_dir.mkdir(exist_ok=True)

# Plot 1: Bending Moment
plt.figure(figsize=(10, 4))
plt.plot(results['x'], results['M']/1000, 'b-', linewidth=2)
plt.xlabel("Position along shaft (mm)")
plt.ylabel("Bending Moment (kN·mm)")
plt.title(f"Bending Moment Diagram\n{material_name}, Load @ {a_load} mm")
plt.grid(True, alpha=0.3)
plt.axvline(x=a_load, color='r', linestyle='--', alpha=0.7, label=f'Load @ x={a_load} mm')
plt.legend()
plt.tight_layout()
plt.savefig(out_dir / "1_bending_moment.png", dpi=150)

# Plot 2: Deflection
plt.figure(figsize=(10, 4))
plt.plot(results['x'], results['y'], 'r-', linewidth=2)
```

```
plt.xlabel("Position along shaft (mm)")
plt.ylabel("Deflection (mm)")
plt.title(f"Deflection Curve\n{material_name}, Load @ {a_load} mm")
plt.grid(True, alpha=0.3)
plt.plot(results['max_deflection_x'], results['max_deflection'], 'ro', markersize=8)
plt.annotate(f'Max: {results["max_deflection"]:.3f} mm',
        xy=(results['max_deflection_x'], results['max_deflection']),
        xytext=(results['max_deflection_x']+20, results['max_deflection']+0.1),
        arrowprops=dict(arrowstyle='->', color='red', alpha=0.7))
plt.tight_layout()
plt.savefig(out_dir / "2_deflection.png", dpi=150)

# Plot 3: Slope
plt.figure(figsize=(10, 4))
plt.plot(results['x'], results['theta']*1000, 'm-', linewidth=2)
plt.xlabel("Position along shaft (mm)")
plt.ylabel("Slope (mrad)")
plt.title(f"Slope Diagram\n{material_name}, Load @ {a_load} mm")
plt.grid(True, alpha=0.3)
plt.axhline(y=0.001*1000, color='g', linestyle='--', label='Bearing limit: 1.0 mrad')
plt.axhline(y=-0.001*1000, color='g', linestyle='--')
plt.axhline(y=0.0005*1000, color='b', linestyle='--', label='Gear limit: 0.5 mrad')
plt.axhline(y=-0.0005*1000, color='b', linestyle='--')
plt.plot(0, results['theta_left']*1000, 'ko', markersize=8, label='Bearings')
plt.plot(L, results['theta_right']*1000, 'ko', markersize=8)
plt.plot(gear_x, results['theta_gear']*1000, 'co', markersize=8, label=f'Gear @ {gear_x} mm')
plt.legend(loc='best')
plt.tight_layout()
plt.savefig(out_dir / "3_slope.png", dpi=150)

# Plot 4: ALL STRESSES ON ONE GRAPH
plt.figure(figsize=(10, 6))

# Create plot with dual y-axes
ax1 = plt.gca()
```

```
ax2 = ax1.twinx()

# Plot normal stresses (bending and von Mises) on left y-axis
line1, = ax1.plot(results['x'], results['sigma_b_surface'], 'b-', label="Bending stress", linewidth=2)
line2, = ax1.plot(results['x'], results['sigma_vm_surface'], 'r-', label="von Mises stress", linewidth=2)

# Plot shear stresses (torsional and transverse) on right y-axis
line3, = ax2.plot(results['x'], results['tau_t_surface'], 'g--', label="Torsional shear", linewidth=2)
line4, = ax2.plot(results['x'], results['tau_s_NA'], 'm:', label="Transverse shear", linewidth=2)

# Mark critical point
ax1.plot(results['crit_x'], results['crit_sigma_vm'], 'ro', markersize=10, markeredgecolor='k')
ax1.annotate(f'Max σ_vm: {results["crit_sigma_vm"]:.1f} MPa',
             xy=(results['crit_x'], results['crit_sigma_vm']),
             xytext=(results['crit_x']+30, results['crit_sigma_vm']+20),
             arrowprops=dict(arrowstyle='->', color='red', alpha=0.7),
             fontsize=10, fontweight='bold')

# Set labels and title
ax1.set_xlabel("Position along shaft (mm)", fontsize=12)
ax1.set_ylabel("Normal Stress (MPa)", fontsize=12, color='blue')
ax2.set_ylabel("Shear Stress (MPa)", fontsize=12, color='green')
ax1.set_title(f"Complete Stress Distribution\n{material_name}, Load @ {a_load} mm, Fillet r={fillet_radius}mm", fontsize=14,
fontweight='bold')

# Color the y-axis labels
ax1.tick_params(axis='y', labelcolor='blue')
ax2.tick_params(axis='y', labelcolor='green')

# Add grid
ax1.grid(True, alpha=0.3)

# Combine legends from both axes
lines = [line1, line2, line3, line4]
labels = [line.get_label() for line in lines]
```

```
ax1.legend(lines, labels, loc='upper left', fontsize=10)

plt.tight_layout()
plt.savefig(out_dir / "4_all_stresses.png", dpi=150)

plt.show()

print(f"\nPlots saved to: {out_dir}/")

# ====================
# SENSITIVITY ANALYSIS
# ====================

run_sensitivity = input("\nRun sensitivity analysis with load moved 10mm right? (y/n): ").lower()

if run_sensitivity == 'y':
    a_load_shifted = a_load + 10.0
    if a_load_shifted > L:
        a_load_shifted = L - 10.0
        print(f"Adjusted shifted load to {a_load_shifted} mm")

    results_shifted = analyze_shaft(material_name + " (shifted load)", S_u, S_y, a_load_shifted, gear_x)

    print("\n" + "="*60)
    print("SENSITIVITY ANALYSIS: LOAD MOVED 10mm RIGHT")
    print("="*60)

    print(f"\nOriginal load @ {a_load} mm:")
    print(f"  Max deflection: {results['max_deflection']:.3f} mm")
    print(f"  Max von Mises: {results['crit_sigma_vm']:.1f} MPa")
    print(f"  Fatigue FoS: {results['n_f']:.2f}")

    print(f"\nShifted load @ {a_load_shifted} mm:")
    print(f"  Max deflection: {results_shifted['max_deflection']:.3f} mm")
    print(f"  Max von Mises: {results_shifted['crit_sigma_vm']:.1f} MPa")
```

```
    print(f"  Fatigue FoS: {results_shifted['n_f']:.2f}")

    deflection_change = ((results_shifted['max_deflection'] - results['max_deflection']) /
                         abs(results['max_deflection'])) * 100
    stress_change = ((results_shifted['crit_sigma_vm'] - results['crit_sigma_vm']) /
                     results['crit_sigma_vm']) * 100

    print(f"\nPercentage changes:")
    print(f"  Deflection: {deflection_change:+.1f}%")
    print(f"  Stress: {stress_change:+.1f}%")

    if abs(stress_change) > 10:
        print("\n  WARNING: Design is highly sensitive to load position!")

print("\n" + "="*60)
print("ANALYSIS COMPLETE")
print("="*60)
```